

59. Un petit air qui joue tout seul

Un Air s'écrit comme une tuile se dessine : une suite de pas, gravée dans le programme.

1. Ce qu'il faut comprendre

Une mélodie, c'est une suite de notes. On pourrait appeler `note()` au bon moment, image après image, avec un compteur ; la console sait le faire toute seule.

Ce qui est nouveau ici : `Air`, et `jouer(voix, AIR, vitesse)`. Un `Air` est une **liste de pas**, comme une tuile est une liste de rangées. Chaque pas est une note et son volume, `"D04 12"`, ou l'un de ces deux signes : `"--"` fait taire la voix, `"=="` laisse la note d'avant continuer (c'est ce qui fait les notes longues).

`jouer(1, CLAIR, 10)` joue l'air sur la voix 1, à raison d'un pas toutes les **10 images**. Ensuite, le programme n'a plus rien à faire : l'air avance tout seul, soixante fois par seconde, même si la boucle du jeu est en retard.

`airFin(1)` rend 1 quand l'air de la voix 1 est arrivé au bout. On s'en sert ici pour écrire « FIN » **une fois** (la variable `fini` évite de le réécrire à chaque image), et A relance l'air depuis le début.

Pour une musique de fond qui ne s'arrête jamais, on ajoute un 4e argument : `jouer(1, CLAIR, 10, 1)` recommence sans fin. Et une deuxième voix peut jouer un autre `Air` en même temps : une basse sous la mélodie.

À toi : écris la suite de la chanson (« mon ami Pierrot ») à la fin de `CLAIR`.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1190 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	74 o	← la console, à l'allumage — la mise en route de la console
main()	191 o	← démarrage — ta fonction
VBlank	11 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AvancerAirs	308 o	← VBlank — fait avancer la musique à chaque image
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte — attend de pouvoir écrire une case
EcrireTexte	10 o	← texte() — écrit un texte, lettre par lettre
CopierTuiles	94 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
LireManette	37 o	← bouton() — lit les huit boutons
Tuiles	208 o	← CopierTuiles — 13 lettres sur 43
Notes	120 o	← AvancerAirs — la table des notes de musique
"AU CLAIR DE LA ..."	19 o	← texte() — un texte gravé
"A: REJOUER"	10 o	← texte() — un texte gravé
"FIN"	3 o	← texte() — un texte gravé
" "	3 o	← texte() — un texte gravé
CLAIR_0	32 o	← jouer() — une table gravée

Lettres gravées : ACDEFIJLNORU:

Pas gravé (jamais appelé) : AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau,

DonneesTransfert

Fonctions du langage appelées :

```
jouer()   x 2   104 o sur place, dans main() + CLAIR_0 (32 o)
texte()   x 4    44 o sur place, dans main() + EcrireTexte (10 o)
airFini() x 1     5 o sur place, dans main()
bouton()  x 1     5 o sur place, dans main() + LireManette (37 o)
image()   x 1     3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : joue « Au clair de la lune » tout seul, écrit FIN à la fin ;
// A la rejoue.
//
// Ce qui est nouveau : Air, une liste de pas de musique.
//  Chaque pas est une note et son volume : D04 12 (do, octave 4, volume 12).
//  == : la note d'avant continue (une note longue).
//  -- : silence.
//  jouer(voix, AIR, vitesse) : joue l'air sur la voix, un pas toutes les « vitesse » images.
//  airFini(voix) : 1 quand l'air de cette voix est arrivé au bout.

// Au clair de la lune : chaque pas dure 10 images.
// « Au clair de la lune, mon a-mi Pier-rot » : 16 pas.

#include <Air>      // un air de musique, note par note
#include <texte>    // écrit un texte à l'écran
#include <jouer>    // joue un air tout seul
#include <airFini>  // dit si un air est fini
#include <bouton>   // lit un bouton de la manette
```

2. UNE MÉLODIE — CLAIR

```
Air CLAIR = {
    "D04 12", "D04 12", "D04 12", "RE4 12",
    "MI4 12", "==",    "RE4 12", "==",
    "D04 12", "MI4 12", "RE4 12", "RE4 12",
    "D04 12", "==",    "==",    "--",
};
```

3. UNE VARIABLE — FINI

```
uint8_t fini = 0;    // 1 quand FIN est déjà écrit
```

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                                // Le jeu commence ici.
    texte(2, 3, "AU CLAIR DE LA LUNE");
    texte(2, 10, "A: REJOUER");

    jouer(1, CLAIR, 10);                    // voix 1, un pas toutes les 10 images.
                                           // L'air avance TOUT SEUL : la boucle n'a rien à faire.

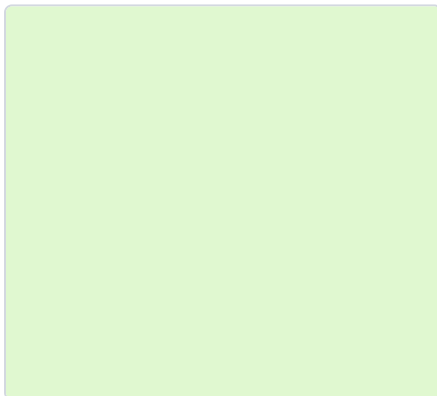
    while (true) {                          // La boucle du jeu :
        image();

        if (airFini(1) && !fini) {          // l'air vient de finir : une seule fois
            texte(8, 6, "FIN");
            fini = 1;                       // pour ne pas réécrire FIN à chaque image
        }

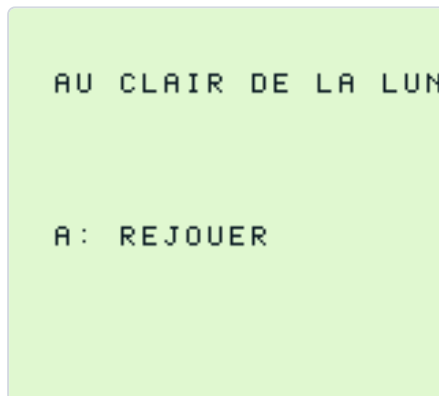
        if (bouton(A) && fini) {            // on ne relance qu'un air fini
            jouer(1, CLAIR, 10);            // depuis le début
            texte(8, 6, " ");              // on efface FIN
            fini = 0;
        }
    }
}
```

3. Ce qui se passe, étape par étape

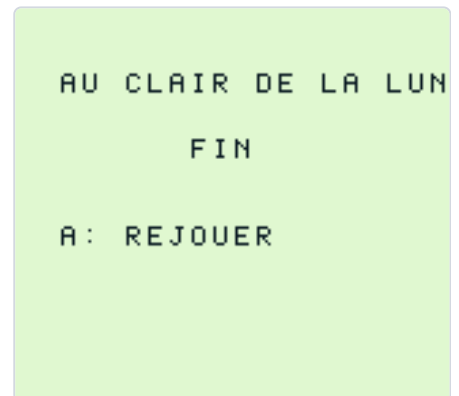
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout de trois secondes — image 180

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.

AU CLAIR DE LA LUN

A : REJOUER

A — l'écran a changé

5. Ce qu'on doit voir

Au clair de la lune joue une fois, puis « FIN » ; A la rejoue.

6. Ce que la console répond

Ces contrôles sont ceux que `verification/verifier-cours.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ l'air commence par un do (DO4, vers 262) (262)

✓ il finit tout seul : FIN

✓ A le relance

← [58. Un son pour chaque action](#)

[sommaire](#)

[60. Faire glisser le décor](#) →